

Quick Reference: Original vs Hybrid ML Plan

What Changed & Why

AT A GLANCE

Aspect	Original Plan	Hybrid Plan	Time Impact
Target Users	Pure individual (B2C)	80% B2B2C + 20% B2C	No change
ML Models	5 core models	Same 5 models	No change
Output Formats	Individual only	Individual + Organization	+2 weeks total
APIs	Single-user endpoints	Multi-tenant endpoints	+1 week
Infrastructure	Basic	Multi-tenant isolation	+1 week
Total Timeline	48 weeks	48 weeks	Same
Engineering Complexity	High	High + System Design	Better for resume

WHAT STAYED THE SAME (70% of Work)

All ML Models: - Receipt OCR (Tesseract + EasyOCR + BERT NER) - Food Recognition (EfficientNet-B4) - Policy NLP (BERT classification) - Time-Series Forecasting (Prophet) - Explainable AI (SHAP, GradCAM) - RL Nudging (DQN)

Training Processes: - Same datasets (SROIE, Food-101, Budget 2026) - Same model architectures - Same evaluation metrics - Same benchmarks

Individual User Features: - Personal dashboards - Receipt scanning - Food logging - Recommendations - All original UX flows

WHAT CHANGED (30% of Work)

1. Receipt OCR - Added Bulk Processing

Original:

POST /ocr/receipt
→ Single receipt image
→ Single user's carbon data

Hybrid:

POST /ocr/receipt *# Same as before*
POST /ocr/receipts/bulk *# NEW: HR uploads 500 receipts*
→ Process in parallel
→ Organization-level summary
→ Department breakdown
→ Individual receipts tagged

Added Work: 2 days (bulk API + aggregation logic)

2. Food Recognition - Added Cafeteria Mode

Original:

POST /food/recognize
→ User photo → prediction → carbon

Hybrid:

POST /food/recognize *# Same as before*
POST /food/cafeteria/menu *# NEW: Pre-recognize cafeteria dishes*
→ Menu stored for day
→ Employees select (no ML needed)
→ Team challenge aggregation

Added Work: 2 days (cafeteria API + team challenges)

3. Forecasting - Added Organization-Level

Original:

forecast_user_emissions(user_id, horizon=30)
→ Individual forecast

Hybrid:

forecast_user_emissions(user_id, horizon=30) *# Same*
forecast_organization_emissions(org_id, horizon=90) *# NEW*
→ Aggregate all employees
→ Department forecasts
→ Comparison analytics

Added Work: 2 days (org aggregation logic)

4. Multi-Tenant Infrastructure - NEW TRACK

What it is: - Data isolation layer (org A can't see org B's data) - Role-based access control (employee vs admin features) - Security decorators on all ML APIs

Why it's needed: - B2B2C = multiple organizations using same system - Data privacy = legal requirement - Shows you understand production deployment

Added Work: 5 days (infrastructure + testing)

5. Admin Analytics APIs - NEW TRACK

What it is: - Organization KPI dashboard endpoint - Department comparison endpoint - Export/reporting endpoint - Team leaderboard endpoint

Why it's needed: - HR/managers need org-wide view - Backend calls these for admin dashboards - Individual features alone don't serve B2B needs

Added Work: 5 days (aggregation APIs)

BEFORE/AFTER COMPARISON

Receipt OCR Example

BEFORE (Pure B2C):

User → Upload receipt photo
→ ML extracts items
→ Shows "Your meal: 2.5 kg CO "
→ Done

AFTER (Hybrid B2B2C):

INDIVIDUAL PATH (same as before):

User → Upload receipt photo
→ ML extracts items
→ Shows "Your meal: 2.5 kg CO "
→ Done

ORGANIZATION PATH (new):

Admin → Upload 500 employee receipts (bulk)
→ ML processes in parallel
→ Organization summary: "Total 1,250 kg CO "
→ Department breakdown: Engineering 450kg, Sales 300kg

- Top categories: Food 600kg, Transport 400kg
- Recommendations: "Consider Meat-Free Mondays"
- Done

Same ML model - two output formats.

Food Recognition Example

BEFORE (Pure B2C):

- User → Photo of biryani
- ML predicts "Biryani" (85% confidence)
- Shows "0.8 kg CO₂ per 100g"
- Logs to user's history

AFTER (Hybrid B2B2C):

INDIVIDUAL PATH (same as before):

- User → Photo of biryani → prediction → logged

CAFETERIA PATH (new):

- Admin → Upload 5 cafeteria dishes (morning)
- ML recognizes once: [Biryani, Dal, Paneer, Salad, Dessert]
- Menu stored for the day

Employee → Open app at lunch

- Sees today's menu (already recognized)
- Selects "Biryani 250g" (no photo, no ML inference)
- Logs instantly (0.8kg × 2.5 servings = 2kg CO₂)
- Contributes to team challenge score

End of Day:

- Team leaderboard updated
- Engineering team: 50 employees, avg 1.2kg/person
- Sales team: 30 employees, avg 1.5kg/person
- Engineering wins the day

Same ML model - used once per dish instead of 50+ times.

WHAT YOU'RE NOT DOING (Scope Control)

Building billing/payment systems Building admin UIs (just APIs) Building email marketing Building HR management features Building complex supply chain (keep Scope 1/2/3 simple) Building two separate ML systems for B2C and B2B

Building ML models that serve both use cases Building organization-level aggregation logic Building multi-tenant data isolation Building admin analytics APIs

TECHNICAL ARCHITECTURE PATTERN

Pattern: One Model → Multiple Views

ML Model (Single Instance)
(e.g., Food Recognition Model)

Individual API	Organization API
Input: Photo	Input: 5 Photos
Output: 1 prediction	Output: Menu (5 predictions)

User Dashboard	Admin Dashboard
----------------	-----------------

Key Insight: Same model, different aggregation levels.

TIME BREAKDOWN: WHERE THE 2 EXTRA WEEKS GO

Addition	Original Time	Hybrid Time	Delta
Receipt OCR	3 days	5 days	+2 days
Food Recognition	4 days	6 days	+2 days
Policy NLP	4 days	4 days	0 days
Forecasting	5 days	7 days	+2 days
XAI + RL	3 days	5 days	+2 days
NEW: Multi-Tenant Infra	0 days	5 days	+5 days
NEW: Admin Analytics	0 days	0 days*	0 days*
Total	~19 days	~32 days	+13 days

Addition	Original Time	Hybrid Time	Delta
----------	---------------	-------------	-------

*Admin Analytics done in Semester 2, parallel with other work

13 days spread over 48 weeks = ~6 hours/week extra work.

Totally manageable.

BENEFITS OF HYBRID APPROACH

For Your Project Grade:

More Impressive: “Built multi-tenant ML system” > “Built individual ML features” **System Design:** Shows you think about deployment, not just algorithms **Real-World:** Demonstrates understanding of actual product requirements **Complexity:** Same ML + architecture → higher technical score

For Your Resume:

Before: > “Built carbon tracking ML models using PyTorch, TensorFlow”

After: > “Architected multi-tenant ML system serving 255K users across B2B2C deployment, handling 500+ concurrent orgs with data isolation, RBAC, and organization-level analytics”

For Panel Questions:

Panel: “Who will use this?” **Before:** “Individuals who care about carbon” (weak) **After:** “80% employer-sponsored (B2B2C), 20% premium individuals. Same ML models serve both markets through multi-tenant architecture.” (strong)

Panel: “How is this different from a class assignment?” **Before:** “We used advanced ML models” (meh) **After:** “We built production-ready multi-tenant infrastructure with data isolation, RBAC, and parallel processing. Our TEME engine has 6 formal benchmarks. We designed for 10K concurrent users.” (impressive)

DELIVERABLES CHECKLIST (Updated)

Semester 1

- ☒ TEME Engine (DONE)
- ☐ Receipt OCR (individual)
- ☐ Receipt OCR (bulk processing)

- ☐ Food Recognition (individual)
- ☐ Food Recognition (cafeteria mode)
- ☐ Policy NLP (already B2B-focused)
- ☐ Forecasting (individual + org)
- ☐ Multi-tenant infrastructure
- ☐ XAI (all models)
- ☐ RL agent (team-aware)

Semester 2

- ☐ Admin analytics APIs
- ☐ FinBERT transaction categorization
- ☐ Mobile app (individual + team features)
- ☐ Blockchain (individual + org verification)
- ☐ Performance optimization
- ☐ Pilot with 2-3 SMEs
- ☐ Documentation
- ☐ Final presentation

COMMON QUESTIONS

Q: “Isn’t this too much scope?”

A: No - we’re adding ~13 days of work (6 hours/week extra) over 48 weeks. The ML models are identical. We’re just adding aggregation logic and multi-tenancy, which are engineering best practices anyway.

Q: “Do we need to build two separate ML systems?”

A: Absolutely not. Same models. Different output formats. Example: Food recognition model runs once on cafeteria menu → serves 50 employees. Same model, more efficient use.

Q: “What if we can’t find SME pilot partners?”

A: The free tier (individual users) gives you 100+ real users. The B2B2C features are still impressive even in demo form. But you’ll likely find 1-2 willing SMEs through your college’s industry connections.

Q: “Is multi-tenant infrastructure too complex?”

A: It’s 5 days of work. Mostly decorators and filters. Critical for any real deployment. Shows maturity. Worth the investment.

YOUR IMMEDIATE NEXT STEPS

This Week (Start Receipt OCR):

1. **Monday-Tuesday:** Follow original guide for individual receipt OCR
 - Environment setup
 - SROIE dataset download
 - Text extraction pipeline
 - Test on 10 sample receipts

2. **Wednesday:** While that works, design bulk API contract

```
# Write the function signature - don't implement yet
def process_receipt_batch(
    receipt_paths: List[str],
    organization_id: str,
    uploaded_by_user_id: str
) -> Dict:
    """TODO: Implement parallel processing"""
    pass
```

3. **Thursday-Friday:** Implement bulk processing

- Use multiprocessing
- Test with 50 receipts
- Measure: time, success rate

Next Week (Food Recognition):

1. **Launch Food-101 training on Colab** (Monday morning - let it run)
2. **While it trains:** Implement cafeteria mode API (Tuesday-Wednesday)
3. **Thursday:** Indian food fine-tuning
4. **Friday:** Test cafeteria workflow end-to-end

FINAL ADVICE

Do: - Follow this hybrid plan - Reuse ML models aggressively - Test multi-tenancy rigorously - Document architecture decisions - Keep individual features excellent

Don't: - Build separate B2C and B2B ML systems - Add business logic beyond aggregation - Gold-plate features - Forget individual users exist - Apologize for hybrid approach (it's smart!)

CONFIDENCE STATEMENT FOR PANEL

“We initially designed for individual users and discovered the market strongly favors employer-sponsored (B2B2C) models. Rather than rebuild, we architected our ML pipelines to serve both use cases from a single codebase using multi-tenant infrastructure and role-based outputs.

The same food recognition model that serves an individual user serves 50 employees through cafeteria mode. The same forecasting model that predicts an individual’s emissions predicts an organization’s emissions by aggregating employee data. This is efficient engineering AND smart product strategy.

We’re demonstrating not just ML competence, but system design maturity - the ability to build platforms, not just features.”

Your project is strong. This plan makes it stronger.